

Projet SecurIT Memory

BINÔME · WINFORMS · C#

SALON DE L'INNOVATION TECH

Vous travaillez pour **SecurIT**, une jeune start-up spécialisée en cybersécurité. L'équipe marketing a besoin d'un mini-jeu interactif pour leur prochain stand au **Salon de l'Innovation Tech**. Votre mission : développer un jeu de cartes **Memory** mettant en scène des icônes de cybersécurité — mots de passe, pare-feu, virus, et bien plus — pour captiver les visiteurs et tester leur mémoire. Ce document est votre guide complet pour mener ce projet à bien, en binôme, avec Visual Studio et C#.

NOTIONS :

- [C# Documentation : Class](#)
- [C# Documentation : Inheritance](#)
- [C# Documentation : Timers](#)
- [C# Documentaion : Bitmap](#)
- [C# Documentation : Graphics](#)
- [Visual Studio documentation : Winform](#)

Contexte & Notions Clés

Votre Mission

Créer un jeu Memory complet et fonctionnel à exposer sur le stand SecurIT lors du salon tech. Le jeu doit être attractif, intuitif et démontrer vos compétences en développement C# et WinForms.

- Projet réalisé **en binôme**
- Rendu via un **lien GitHub** sur Moodle
- Accompagné d'un **README.md** détaillé
- Utilisation obligatoire de **Visual Studio**

Notions à Maîtriser

Classes C#

Conception orientée objet avec la classe `Carte`

Listes & Tableaux

Gestion des collections de cartes et des scores

Timers C#

Gestion du délai de retournement et du chronomètre

WinForms

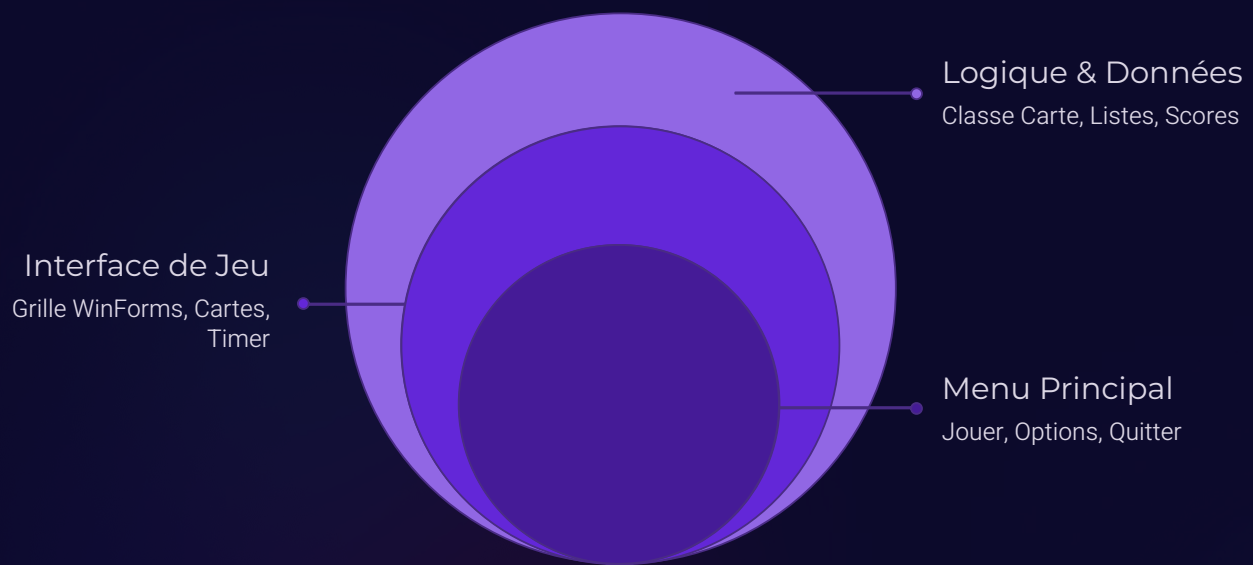
PictureBox, Boutons, Labels et mise en page graphique



Notion optionnelle : Connexions SQL pour le tableau des scores (partie bonus).

Architecture du Projet

Avant d'écrire la moindre ligne de code, il est essentiel de comprendre comment les différentes composantes de votre projet s'articulent. Un projet WinForms Memory bien structuré repose sur une séparation claire entre la logique de jeu, l'interface graphique et la gestion des données.



Cette architecture en trois couches vous permettra de travailler en parallèle avec votre binôme : l'un peut s'occuper de l'interface WinForms pendant que l'autre développe la logique de la classe `Carte`. Une bonne séparation des responsabilités est aussi un critère évalué dans la grille de notation.

Partie 1 — La Classe Carte

Le cœur de votre jeu repose sur une conception orientée objet rigoureuse. La classe `Carte` est l'élément central que vous devez implémenter avec soin. Elle encapsule toutes les informations nécessaires à chaque carte du jeu et définit son comportement.

ID / Valeur de Paire

Chaque carte possède un identifiant numérique qui lui permet d'être associée à sa carte jumelle. Deux cartes partageant le même ID forment une paire valide.

Image Associée

Une propriété de type `Image` ou `string` (chemin) qui stocke l'icône de cybersécurité à afficher quand la carte est révélée (virus, cadenas, pare-feu...).

État de la Carte

Une énumération à trois valeurs : `Cachée` (face verso), `Révélée` (temporairement visible), `Trouvée` (paire identifiée, reste visible).



Bonne pratique POO : Utilisez des propriétés avec `get` et `set` pour respecter le principe d'encapsulation. Ne laissez pas vos champs en accès public direct. Cela sera évalué dans le critère "Conception Orientée Objet".

En plus de la classe `Carte`, pensez à créer une classe ou un gestionnaire `JeuMemory` qui maintient la liste de toutes les cartes (via un `List<Carte>`), gère le mélange aléatoire au démarrage, et orchestre les interactions entre les cartes sélectionnées.

Partie 1 — Interface WinForms & Grille



Le Menu Principal

Votre application doit démarrer sur un menu clair et professionnel comportant trois boutons fonctionnels :

→ Jouer

Lance une nouvelle partie en initialisant et mélangeant les cartes.

→ Options

Permet de choisir la taille de la grille (4×4, 6×6) ou le niveau de difficulté.

→ Quitter

Ferme proprement l'application.



La Grille de Jeu

Générée dynamiquement dans le formulaire WinForms, la grille utilise des `PictureBox` pour représenter chaque carte. Chaque `PictureBox` doit être liée à un objet `Carte`.

- Afficher le **nombre d'essais** courant via un `Label`
- Afficher le **chronomètre en cours** mis à jour en temps réel
- Utiliser des images thématiques **cybersécurité** pour les faces
- Une image **dos de carte** uniforme pour les faces cachées

Partie 1 — Logique de Jeu & Timers

La logique de gameplay est l'âme de votre projet. Elle doit être à la fois robuste et réactive pour offrir une expérience fluide aux visiteurs du salon. Les **Timers C#** jouent un rôle double et critique dans ce projet.



📌 **⚠ Piège fréquent** : Pendant que le Timer de délai est en cours (cartes non-correspondantes visibles), bloquez les clics sur les autres cartes ! Sinon, le joueur peut sélectionner une 3ème carte avant le retournement, ce qui corrompt la logique de jeu.

Partie Bonus — Allez Plus Loin !

La partie bonus est optionnelle mais constitue une excellente opportunité de vous démarquer et d'impressionner le jury. Ces fonctionnalités supplémentaires témoignent de votre maîtrise des concepts avancés et de votre créativité en tant que développeurs.



Leaderboard SQL

Connectez votre jeu à une base de données SQL locale pour sauvegarder les scores (nom, temps, essais) et afficher un classement dynamique depuis le menu principal.



Effets Sonores

Ajoutez des sons immersifs : un clic au retournement, un son de succès pour une paire trouvée, et une fanfare de victoire en fin de partie.



Thèmes de Cartes

Proposez plusieurs thèmes visuels au choix dans les Options : **Matériel** (RAM, CPU), **Logiciel** (OS, apps) et **Cryptographie** (clés, algorithmes).



Mode Hardcore

Un mode de jeu extrême où les cartes non-trouvées échangent leurs positions aléatoirement toutes les **30 secondes**, mettant la mémoire à rude épreuve.

Grille d'Évaluation — 20 Points

Votre projet sera noté sur 20 points, répartis équitablement entre la partie technique et la présentation orale. Prenez connaissance de chaque critère dès le début du projet pour orienter vos efforts au bon endroit.

⚙️ Partie Technique & Code — /10

Critère	Pts	Focus
Fonctionnalités de base	3	Jeu fonctionnel, menu, timer, victoire
Conception Orientée Objet	3	Classe Carte, encapsulation, structure
Interface WinForms	2	UI propre, images, compteurs lisibles
Qualité Code & Git	2	Indentation, commentaires, README.md

🎤 Présentation Orale — /10

Critère	Pts	Focus
Scénarisation & Contexte	2	Contexte SecurIT, besoins exposés
Clarté de l'expression	2	Vocabulaire adapté, fluidité
Équilibre & Maîtrise	3	50/50 parole, comprendre le code de l'autre
Démonstration & Questions	2	Démo live, réponses précises au jury
Posture professionnelle	1	Tenue, dynamisme, sérieux

📌 🏆 **Conseil stratégique :** Les critères "Conception Orientée Objet" (3 pts) et "Équilibre & Maîtrise" (3 pts) sont les plus pondérés. Investissez du temps dans une classe Carte propre ET assurez-vous que chaque membre du binôme comprend l'intégralité du code.

Conseils & Bonnes Pratiques

Organisation GitHub dès le Jour 1

Créez votre dépôt GitHub dès le début du projet. Commitez régulièrement avec des messages clairs ("*Ajout classe Carte*", "*Fix timer délai retournement*"). Un historique de commits régulier témoigne d'un travail progressif et sérieux. Rédigez votre `README.md` au fil du projet, pas à la dernière minute.

Division du Travail en Binôme

Répartissez les tâches intelligemment : l'un développe la logique (classe `Carte`, algorithmes de jeu, `Timer`) pendant que l'autre conçoit l'interface WinForms (grille, menu, Labels). Puis échangez pour que chacun comprenne le code de l'autre avant la présentation orale.

Tester Progressivement

N'attendez pas d'avoir tout codé pour tester. Testez chaque fonctionnalité indépendamment : d'abord l'affichage d'une carte, puis le retournement, puis la comparaison de paires, puis le `Timer`. Les bugs sont bien plus faciles à isoler en testant par petites étapes.

Préparer la Présentation Orale

Le jury évalue autant la présentation que le code. Préparez un discours d'ouverture qui contextualise SecurIT et le besoin métier. Répétez la démo live sur l'ordinateur de présentation. Anticipez les questions : "*Pourquoi avez-vous choisi une classe énumération pour les états ?*"

Récapitulatif & Checklist de Rendu

Avant de soumettre votre lien GitHub sur Moodle, vérifiez que chaque point de cette checklist est coché. Un projet complet et bien livré, c'est déjà la moitié du chemin vers une excellente note.

✓ Checklist Technique

- Le jeu se lance sans erreur depuis Visual Studio
- Le menu avec Jouer / Options / Quitter est fonctionnel
- La classe `Carte` est implémentée avec encapsulation
- La grille se génère et les cartes se mélangent aléatoirement
- Le Timer de délai fonctionne (retournement des non-paires)
- Le chronomètre et le compteur d'essais s'affichent
- La victoire est détectée et affichée correctement
- Le code est commenté et bien indenté

✓ Checklist Rendu & Oral

- Dépôt GitHub créé et lien soumis sur Moodle
- Fichier `README.md` détaillé présent dans le repo
- Les deux membres du binôme comprennent le code complet
- Le discours d'ouverture (contexte SecurIT) est préparé
- La démo live est testée sur l'ordinateur de présentation
- Les questions techniques anticipées et travaillées
- Tenue professionnelle prévue pour le jour J
- Temps de parole équilibré 50/50 répété

20

Points au Total

10 technique + 10 oral

2

Membres par
Équipe

Travail en binôme
obligatoire

3

Pts POO

Critère le plus pondéré
côté code

4

Bonus Possibles

SQL, thèmes, sons,
mode Hardcore